BER Viewer

Desktop-Anwendung zum Anzeigen und Bearbeiten von BER/ASN.1-Dateien, spezialisiert auf **ETSI LI PS-PDU** und **3GPP Lawful Intercept** Formate.

Voraussetzungen

- **Node.js** ≥ 18 (inkl. npm)
- **ASN.1-Schema-Verzeichnis** `asn1_patched/` — muss im selben Ordner wie `package.json` liegen (31 `.asn`/`.asn1`-Dateien)

Installation & Start

```
# Abhängigkeiten installieren (einmalig)
npm install

# Anwendung starten
npm start
```

Build

Windows Installer

```
npm run build:win
```

→ `dist\BER Viewer Setup x.x.x.exe`
Oder ohne Installation: `dist\win-unpacked\BER Viewer.exe`

Linux AppImage (Ubuntu / Linux Mint)

```
# Node.js 20 LTS installieren (falls noch nicht vorhanden)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# System-Abhängigkeiten für Electron
sudo apt install -y libgconf-2-4 libatk1.0-0 libatk-bridge2.0-0 \
    libgdk-pixbuf2.0-0 libgtk-3-0 libgbm-dev libnss3 libasound2 \
    fuse libfuse2

# Build
npm install
npm run build:linux
```

→ `dist/BER Viewer-x.x.x.AppImage`

```
chmod +x "dist/BER Viewer-x.x.x.AppImage"
./"dist/BER Viewer-x.x.x.AppImage"

# Falls FUSE-Fehler:
./"dist/BER Viewer-x.x.x.AppImage" --no-sandbox
```

Projektstruktur

```
ber_viewer_electron/
├── package.json
├── asn1_patched/          ← 31 ASN.1-Schemadateien (neben package.json!)
└── src/
    ├── main.js            ← Electron-Hauptprozess, BER-Parser, IPC
    ├── preload.js         ← IPC-Bridge zwischen Main und Renderer
    ├── renderer.js        ← UI, Tree-Rendering, Edit-Dialog, SMS-Decoder
    ├── index.html         ← Toolbar, Suchfeld, Baumansicht
    └── style.css          ← Dark Theme
```

Unterstützte Dateiformate

Der Viewer erkennt den Dateityp automatisch anhand der ersten BER-Bytes und der eingebetteten OID:

Dateiformat	Erkennung	Beispiel-Dateiname
5G PS-PDU (TS 33.128)	`0x30` + OID `0.4.0.2.2.5.x`	`*.li_ps_pdu_5G.hi2`
EPS PS-PDU (HI2 r14/r15)	`0xa1` + OID `0.4.0.2.2.4.8.x`	`*.li_ps_pdu_Not5G.hi2`
LI PS-PDU (ETSI 102 232)	`0x30` + OID `0.4.0.2.2.5.x`	`*.li_ps_pdu.hi2`
UmtsCS IRI iRI-Begin	`0xa1` + OID `0.4.0.2.2.4.3.x`	`D2AE*`, `E2GG*`
UmtsCS IRI iRI-Continue	`0xa3` + OID `0.4.0.2.2.4.3.x`	`D2AE*` (Continue)
UmtsCS IRI iRI-Report	`0xa4` + OID `0.4.0.2.2.4.3.x`	`E2AG*`
UmtsCS IRI (wrapped)	`0xa0`	—

Bedienung

Toolbar

Schaltfläche	Tastenkürzel	Funktion

Schaltfläche	Tastenkürzel	Funktion
Open	`Ctrl+O`	Datei öffnen (auch per Drag & Drop)
Expand	—	Alle Knoten aufklappen
Collapse	—	Alle Knoten zuklappen
Save As	`Ctrl+S`	Als BER-Datei speichern (re-serialisiert)
Export TXT	—	Baum als Text exportieren (Format 1 oder 2)
Suche	—	Feldname oder Wert filtern

Navigation

- **Klick** auf einen Knoten → Details rechts (Decoded Value + Hex Dump)
- **Klick** auf ▶ / ▼ → Knoten auf-/zuklappen
- **Doppelklick** auf einen Blattwert → Wert bearbeiten (Text oder Hex)
- **Rechtsklick** → Kontextmenü

Kontextmenü

Eintrag	Beschreibung
🖊 Bearbeiten	Wert direkt editieren
📋 Wert kopieren	Decoded Value in Zwischenablage
📋 Hex kopieren	Rohdaten als Hex-String
⊞ Aufklappen	Teilbaum aufklappen
⊟ Zuklappen	Teilbaum zuklappen
📱 SMS dekodieren	SMS-PDU-Inhalt anzeigen (nur bei `content`-Feldern)

Zuletzt geöffnete Dateien

`Datei → Zuletzt geöffnet` — beim Wechsel fragt der Viewer bei ungespeicherten Änderungen nach.

SMS-Decoder

Rechtsklick auf ein `content [4]`-Feld in `sMS-Contents` → 📱 **SMS dekodieren**.

Typ	Unterstützung
SMS-DELIVER	Absender, Zeitstempel, Text
SMS-SUBMIT	Empfänger, Text
SMS-STATUS-REPORT	Sendezeit, Zustellzeit, Statuscode
GSM 7-Bit	Standardzeichensatz (Deutsch, Englisch ...)

Typ	Unterstützung
8-Bit (Latin-1)	Erweiterter Zeichensatz
UCS-2	Unicode (Arabisch, Chinesisch ...)
Multipart (UDH)	Teil- und Gesamtanzahl werden angezeigt

Bearbeitung & Speichern

- Geänderte Felder werden **orange** markiert; der Fenstertitel zeigt `*`.
- **Save As** re-serialisiert den vollständigen Baum als korrektes BER mit aktualisierten Längenfeldern.
- **Export TXT** bietet zwei Formate:
  - **Format 1** — eingerückte Baumdarstellung
  - **Format 2** — tabellarisch mit Offset, Tag und Wert

ASN.1-Schema-Auflösung

Beim Start werden alle `*.asn` / `*.asn1`-Dateien aus `asn1_patched/` geladen und zu Tag-Maps verarbeitet. Zusätzlich gibt es hartcodierte **virtuelle Typen** für Felder, die in der ASN.1 als anonyme Inline-SEQUENCEs definiert sind:

Virtueller Typ	Felder
`EpsPartyIdentity`	`imei`, `imsi`, `msISDN`, `sip-uri`, `nai`
`UmtsHI2PartyIdentity`	`imei`, `imsi`, `callingPartyNumber`, `msISDN`, `e164-Format`
`GsmGeoCoordinates`	`latitude`, `longitude`, `mapDatum`, `azimuth`
`UtmCoordinates`	`utm-East`, `utm-North`
`SmsContents`	`initiator`, `transfer-status`, `other-message`, `content`
`MessagingCC`	`event-identifier`, `content-type`, `content`
`IPMMIRILocation`	`umtsHI2Location`, `epsLocation`, `wlanLocation`
`SNSSAI`	`sliceServiceType`, `sliceDifferentiator`
`TAI`	`pLMNID`, `tAC`, `nID`
`CCContents`	`payloadDirection`, `messagingCC`, `iPCC`, ...

Gemessene Label-Abdeckung (30 Testdateien)

Dateityp	Felder	Labelquote
5G PS-PDU (`li_ps_pdu_5G`)	~60	**92–96 %**
EPS PS-PDU (`li_ps_pdu_Not5G`)	~34–61	**88–95 %**
LI PS-PDU (`li_ps_pdu`)	~64–75	**92–96 %**

Dateityp	Felder	Labelquote
UmtsCS IRI (`D2AE`, `D2DE`, `E2AG`, `E2GG`)	~37–56	**64–95 %**
CC-Payload/Messaging (`DT*`)	~26	**85 %**
Gesamt (1549 Knoten)	—	**≥ 99 %**

Bekannte Einschränkungen

- Felder ohne ASN.1-Kontextmarkierung (manche SEQUENCE-OF-Elemente) werden mit generischem `SEQUENCE`-Label angezeigt — die Daten sind vollständig sichtbar.
- Sehr große Dateien (> 50 kB) können das Rendern verlangsamen.
- Code-Signierung ist deaktiviert (`CSC_IDENTITY_AUTO_DISCOVERY=false` in `package.json`).

Entwicklung (ohne Build)

```
# Direkt starten ohne Installer
npm start
```